Atividade_05 - Livro AVR e Arduino – Técnicas de Projeto
Capítulo: 19 (Conversor Analógico-Digital)

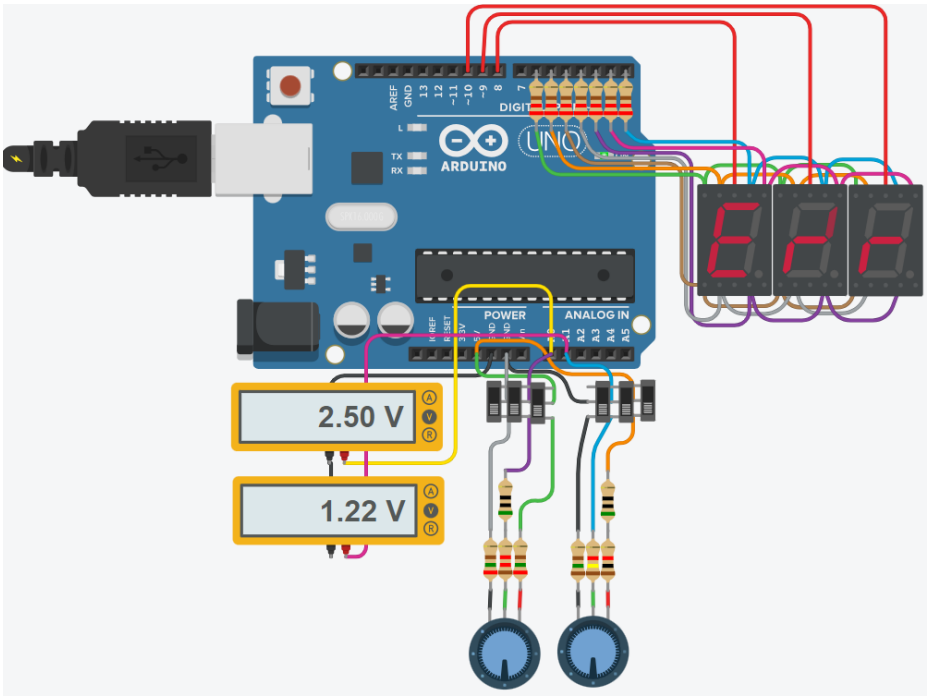
Título: Interfaceando um pedal de acelerador com Arduino

Objetivos: O objetivo é fazer a leitura com sucesso de um pedal de acelerador. Também devemos conseguir identificar se o pedal está funcionando corretamente ou se a ligação com ele está íntegra.

Nesta prática utilizaremos o Tinkercad para simular um circuito simples usando o microcontrolador Atmega328, utilizado nas placas Arduino UNO.

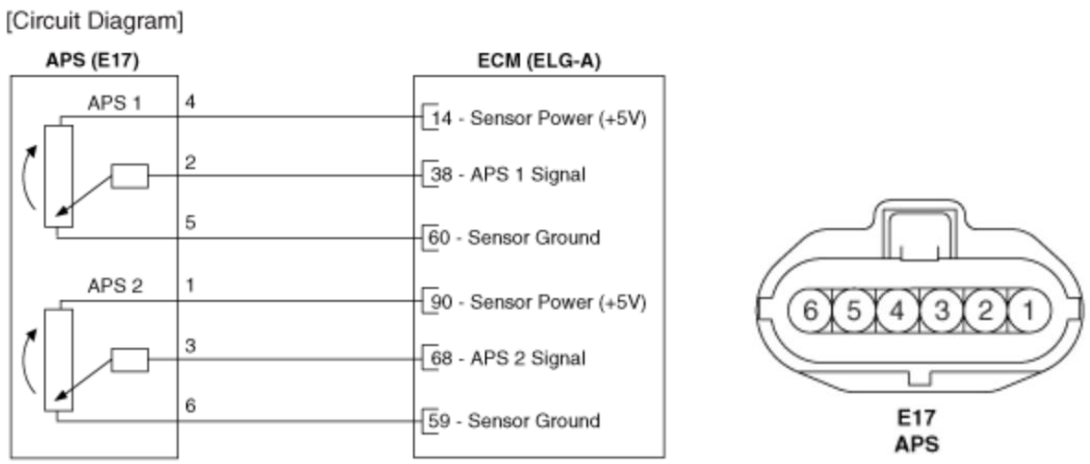
1. Procedimentos:

1. Copie o projeto fornecido em https://www.tinkercad.com/things/9phPMDYNTwP?sharecode=kNXb_CGvUZMdb2VMPM_A5KZaDfrdU5mzx55RtWVBDvs:

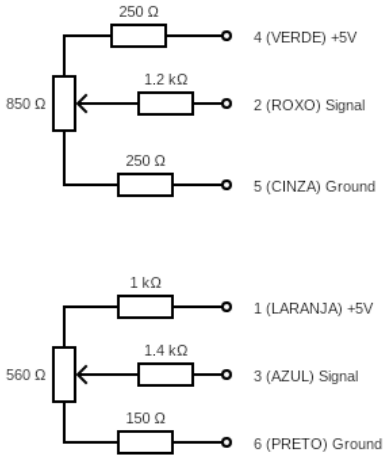


Os dois potenciômetros representam os potenciômetros que normalmente são empregados em pedais de acelerador eletrônico. Eles são sempre utilizados aos pares para permitir a identificação de falhas no componente. A razão entre eles normalmente é de 2:1, o que ajuda a identificar falhas, inclusive na ligação.

O pedal simulado é o 32726-3k100 com o sensor DJQ0366A-3L100, representado na figura que segue:



O diagrama esquemático equivalente dele é o que segue na figura:



No modelo de projeto há um código que faz a leitura de um dos potenciômetros. Ele exibe o valor bruto da leitura no monitor serial (via porta serial) para facilitar a leitura. Anote estes valores máximos e mínimos para uso posterior. Leia também os valores para o segundo potenciômetro.

2. Você deve exibir a porcentagem de aceleração nos displays de 7 segmentos fornecidos. Na posição mínima o valor a ser exibido é 0, e na posição máxima, 100. O valor exibido é a média entre os dois potenciômetros. Caso a diferença entre os dois exceda 10%, você deve indicar um erro como exibido no display no modelo.

3. Quando qualquer fio for quebrado (indicado pelos interruptores deslizantes), você deve exibir um erro no display.

RESTRIÇÃO DE PROJETO a: O circuito utilizado no projeto deve ser o fornecido, podendo as mudanças ocorrerem somente em relação ao software. Não devem haver delays (espera ociosa), exceto para o caso da simulação rodar mais rapidamente.

RESTRIÇÃO DE PROJETO b: A leitura do ADC deve ser feita utilizando-se interrupções. A troca do canal deve ser feita dentro da rotina de interrupção. Não utilizar espera ocupada na leitura do ADC! Apagar toda e qualquer referência a porta serial no código final.

Atenção: escrever um código que favoreça a leitura. Variáveis, funções, e afins devem ter nomenclatura pertinente ao seu papel no código. Variáveis genéricas como i,j somente como auxiliares de contagem em laços. Arrays com nomes de um único caractere, como X[] não são aceitáveis.

4. Cole o código fonte do microcontrolador ao final deste arquivo.

OBRIGATÓRIO PARA AVALIAÇÃO: torne seu projeto compartilhado por link (não público) e cole o link para ele aqui:
https://www.tinkercad.com/things/03o6hYBlsws-copy-of-adc-cf-2026-pratica-5-modelo?sharecode=JiqkY-uppgWHn3PkcER0IRv3nUPUn_cTYMaJwbOgNCA

Também coloquei o link no código fonte :)

Anexe o código fonte em um arquivo separado.

ATENÇÃO: Documente seu código. Cada linha/bloco deve deixar explícito o seu papel.

RÚBRICA:
Lógica da programação e funcionamento (desde que atendendo às restrições de projeto): 100%

Valor desta atividade na média: 0.9

6. Notas extras:
6.1 Multiplexar o chaveamento dos displays dentro da rotina de interrupção do ADC (atenção, explicar por que isto é viável; chavear os displays próximo a 1Khz): 0.1 na média.

```
//#CODIGO:


const int POT1_MIN = 833;
const int POT1_MAX = 190;
const int POT2_MIN = 413;
const int POT2_MAX = 87;


unsigned char digitoAtivo = 0;
unsigned long ultimaAtualizacaoDisplay = 0;

// Mapa de bits do display: {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, E, r}
unsigned char mapaDisplay[] = {0x40, 0x79, 0x24, 0x30, 0x19, 0x12, 0x02, 0x78, 0x80,
0x18, 0x06, 0x2F};
unsigned char valoresDosDigitos[3] = {10, 11, 11}; // Inicia exibindo 'Err'

// Variáveis atualizadas na Interrupção (devem ser 'volatile')
volatile int leituraBrutaPot1 = 0;
volatile int leituraBrutaPot2 = 0;
volatile unsigned char canalAdcAtual = 0;
volatile bool novoParDeLeiturasPronto = false;

void setup()
{
  DDRD |= 0b01111111; // Define Pinos do Display como saída (Segmentos)
  DDRB |= 0b00000111; // Define Pinos do Display como saída (Dígitos)

  DDRC &= ~(1<<PC0);  // Configura PC0 (A0) como entrada
  DDRC &= ~(1<<PC1);  // Configura PC1 (A1) como entrada

  // Configuração do ADC
  ADMUX &= ~((1<<REFS0)|(1<<REFS1));
  ADMUX |= (1<<REFS0); // Tensão de referência em AVCC
```

```
  // Seleciona o canal inicial (0)
  ADMUX &= 0b11110000;

  ADCSRB = 0;

  // Configura o Prescaler para 128
  ADCSRA &= 0b11111000;
  ADCSRA |= (1<<ADPS2)|(1<<ADPS1)|(1<<ADPS0);

  // Habilita a Interrupção e o Módulo ADC
  ADCSRA |= (1<<ADIE);
  ADCSRA |= (1<<ADEN);

  // Dispara a primeira conversão
  ADCSRA |= (1<<ADSC);
}

void loop()
{

  //  Lógica de Atualização (Multiplexação) dos Displays
  if (millis() > (ultimaAtualizacaoDisplay + 5)) {
    ultimaAtualizacaoDisplay = millis();
    digitoAtivo++;

    if (digitoAtivo > 2) {
      digitoAtivo = 0;
    }

    // Ativa o dígito correspondente e joga o padrão no PORTD
    PORTB = (PORTB & 0b11111000) | (1 << digitoAtivo);
    PORTD = mapaDisplay[valoresDosDigitos[digitoAtivo]];
  }


  //  Processamento dos Dados do Acelerador

  if (novoParDeLeiturasPronto) {
    novoParDeLeiturasPronto = false;

    // Pausa interrupções brevemente para copiar as variáveis atômicas
    cli();
    int pot1 = leituraBrutaPot1;
    int pot2 = leituraBrutaPot2;
    sei();

    // CORREÇÃO: Detecção de "Fio Quebrado" à prova de limites invertidos
    int limiteInf1 = min(POT1_MIN, POT1_MAX) - 10;
    int limiteSup1 = max(POT1_MIN, POT1_MAX) + 10;
    bool fioQuebrado1 = (pot1 < limiteInf1) || (pot1 > limiteSup1);

    int limiteInf2 = min(POT2_MIN, POT2_MAX) - 10;
    int limiteSup2 = max(POT2_MIN, POT2_MAX) + 10;
    bool fioQuebrado2 = (pot2 < limiteInf2) || (pot2 > limiteSup2);

    if (fioQuebrado1 || fioQuebrado2) {
      valoresDosDigitos[0] = 10; // E
      valoresDosDigitos[1] = 11; // r
      valoresDosDigitos[2] = 11; // r
```

```
    }
    else {
      // Converte leituras brutas em porcentagem (0 a 100)
      long pctAceleracao1 = map(pot1, POT1_MIN, POT1_MAX, 0, 100);
      long pctAceleracao2 = map(pot2, POT2_MIN, POT2_MAX, 0, 100);

      // Limita os valores entre 0 e 100 para evitar números negativos ou acima de 100
      pctAceleracao1 = constrain(pctAceleracao1, 0, 100);
      pctAceleracao2 = constrain(pctAceleracao2, 0, 100);

      // Detecção de falha por divergência de valores (Restrição de 10%)
      if (abs(pctAceleracao1 - pctAceleracao2) > 10) {
        valoresDosDigitos[0] = 10; // E
        valoresDosDigitos[1] = 11; // r
        valoresDosDigitos[2] = 11; // r
      }
      else {
        // Funcionamento normal: calcula a média e atualiza o display
        int mediaAceleracao = (pctAceleracao1 + pctAceleracao2) / 2;

        valoresDosDigitos[0] = (mediaAceleracao / 100) % 10; // Centenas
        if (mediaAceleracao < 100) valoresDosDigitos[0] = 0; // Apaga o zero à esquerda do
100, se preferir

        valoresDosDigitos[1] = (mediaAceleracao / 10) % 10;  // Dezenas
        valoresDosDigitos[2] = mediaAceleracao % 10;         // Unidades
      }
    }
  }

  delay(1); // Mantido apenas para a simulação
}

// Rotina de Serviço de Interrupção do ADC

ISR(ADC_vect)
{
  if (canalAdcAtual == 0) {
    leituraBrutaPot1 = ADC;
    canalAdcAtual = 1;
    ADMUX = (ADMUX & 0b11110000) | 1; // Troca para o canal 1 (A1)
  } else {
    leituraBrutaPot2 = ADC;
    canalAdcAtual = 0;
    ADMUX = (ADMUX & 0b11110000) | 0; // Retorna para o canal 0 (A0)
    novoParDeLeiturasPronto = true;   // Avisa o loop principal que o par foi lido
  }

  // converte imediatamente após capturar o dado
  ADCSRA |= (1<<ADSC);
}
```